

Handwritten Digit Classification System Powered by the FPGA

IC Project 1 - ETIN25

Grupp 8

Qing Mu & Marcel Martens & Melwin Bauer

Handledare: Lina Tinnerberg

LTH
Lunds Universitet
Sverige
October 30, 2026

Contents

Design Specification **3**

Purpose . . . 3

Scope and System Overview . . . 3

Functional Requirements . . . 4

System-Level Architecture and Data Flow . . . 5

CNN Accelerator Core Architecture . . . 6

Design Choices . . . 7

a) Input Data Handling and Quantization . . . 7

b) Line Buffer and Sliding-Window Generation . . . 7

c) Convolution Parallelism . . . 7

d) Max-Pooling and Flattening . . . 7

e) Flatten . . . 8

f) Fully Connected Layer Parallelism . . . 8

g) Fixed Weights, Biases, and Quantization Constants . . . 8

h) Grade-5 Multi-Accelerator Routing . . . 8

RTL Module Structure and Integration . . . 9

External Interfaces . . . 9

a) Top-Level Module . . . 9

b) AXI-Lite Control and Status Interface . . . 9

c) AXI-Stream Input Interface . . . 10

d) AXI-Stream Output Interface . . . 10

e) Core-Level Ready/Valid Interface . . . 11

Constraints and Limitations . . . 11

Verification Plan and Acceptance Criteria . . . 12

Design Specification

Purpose

The purpose of this design is to implement a hardware accelerator for handwritten digit classification using a quantized Convolutional Neural Network (CNN). The accelerator receives grayscale MNIST images from the Zynq processing system through an AXI DMA, performs CNN inference in FPGA programmable logic, and returns ten classification scores back to the processing system.

The initial project target was to design a single-core CNN accelerator that could fit within the resource constraints of a PYNQ-Z1 board. This baseline constraint influenced the architecture, especially the selected level of convolution and fully connected layer parallelism. However, the final grade-5 objective required multiple accelerator cores operating in parallel. Therefore, the final multi-accelerator implementation and the reported timing, resource, performance, and power results are based on the RFSoC platform used in the course.

The design is intended to reduce the amount of computation performed by the processing system and to demonstrate a hardware-oriented implementation of a CNN inference pipeline. The final grade-5 system supports multiple images in one DMA transfer and multiple CNN accelerator cores connected through input and output routers. This allows the system to process a continuous package of images and improve throughput compared with a single-core implementation.

Scope and System Overview

The complete grade-5 top-level module is `router_top_g5`. It provides the external AXI-Lite control interface and the AXI-Stream interfaces connected to the DMA. The router works as a wrapper to the accelerator, handling incoming signals aswell as the dataflow to and from parallel accelerators.

The module `cnn_integration_top` implements one complete CNN accelerator core. The grade-5 top-level module instantiates multiple `cnn_integration_top` cores and connects them through `router_input_multi` and `router_output_multi` that handle the internal dataflow.

The accelerator implements the fixed CNN structure shown in Table 1.

Table 1: CNN layer dimensions and data formats.

Stage	**Output shape**	**Description**
Input image	$28 \times 28 \times 1$ uint8	One grayscale MNIST image.
Input quantization	$28 \times 28 \times 1$ int8	Input pixels are converted from uint8 to the signed int8 representation used by the CNN datapath.
Convolution	$26 \times 26 \times 32$ int8	3×3 convolution, stride 1, VALID padding, and 32 output channels.
Max pooling	$13 \times 13 \times 32$ int8	2×2 max pooling with stride 2.
Flatten	5408 int8	The downsampled feature map is flattened before the fully connected layer.
Fully connected	10 int8	The final layer produces ten class scores, one for each digit class.

The RTL does not perform the final argmax operation. Instead, the accelerator returns ten signed 8-bit class scores. Software obtains the predicted digit by selecting the index of the maximum score.

Functional Requirements

Table 2 lists the functional requirements of the accelerator. The requirements are organized according to the grade-3, grade-4, and grade-5 project targets. Since the final goal of this design is a grade-5 implementation, the grade-3 and grade-4 requirements are treated as baseline requirements, while the grade-5 requirements extend the design with multi-accelerator execution.

Table 2: Functional requirements.

ID	Target	Priority	Description
FR-01	Grade 3	Must	Provide a software behavioural model that acts as the golden reference for the quantized CNN computation.
FR-02	Grade 3	Must	Implement a hardware accelerator for MNIST handwritten digit classification using a quantized CNN datapath.
FR-03	Grade 3	Must	Accept 28×28 unsigned 8-bit grayscale images as input data.
FR-04	Grade 3	Must	Convert the input image from uint8 representation to the signed int8 representation used internally by the CNN datapath.
FR-05	Grade 3	Must	Perform 3×3 convolution with stride 1 and VALID padding, producing a $26 \times 26 \times 32$ int8 output feature map.
FR-06	Grade 3	Must	Perform 2×2 max pooling with stride 2, producing a $13 \times 13 \times 32$ int8 feature map.
FR-07	Grade 3	Must	Flatten the pooled feature map into a 5408-element int8 feature vector.
FR-08	Grade 3	Must	Perform a fully connected layer from 5408 input features to 10 output classes.
FR-09	Grade 3	Must	Output ten int8 class scores for each input image. The softmax layer is not implemented in RTL, and the final argmax and removal of padding may be performed in software.
FR-10	Grade 3	Must	Use int8 weights and activations with wider signed accumulators for convolution and fully connected computations.
FR-11	Grade 3	Must	Apply the required bias addition, requantization, zero-point handling, and clamping so that RTL results match the quantized behavioural model as closely as possible.
FR-12	Grade 3	Must	Package the accelerator as a custom Vivado IP that can be connected to the Zynq processing system through AXI DMA.
FR-13	Grade 3	Must	Use AXI-Stream ready/valid handshaking on the accelerator input and output data interfaces.
FR-14	Grade 3	Must	Provide a Vivado block design containing the Zynq processing system, AXI DMA, and the accelerator IP.
FR-15	Grade 3	Must	Provide PYNQ software that can send one image through DMA and receive one corresponding result.
FR-16	Grade 3	Must	Produce classification results that mostly match the behavioural model for the selected test images.
FR-17	Grade 4	Must	Support multiple input images in a single DMA transfer.
FR-18	Grade 4	Must	Return multiple output results for one input package, with one result corresponding to each input image.
FR-19	Grade 4	Must	Include a router that can handle different package sizes, meaning different numbers of images per DMA transfer.
FR-20	Grade 4	Must	Allow the package size to be changed at runtime from PYNQ through an AXI-Lite control register.
FR-21	Grade 4	Must	Provide a debugging strategy suitable for a large data stream, for example by comparing RTL output files or stage outputs against the behavioural model.
FR-22	Grade 5	Must	Support multiple CNN accelerator cores operating in parallel.

ID	Target	Priority	Description
FR-23	Grade 5	Must	Keep a single external AXI DMA input stream and a single external AXI DMA output stream for the complete multi-accelerator system.
FR-24	Grade 5	Must	Use the input router to divide complete input images across the available accelerator cores.
FR-25	Grade 5	Must	Use the output router to collect results from all accelerator cores and combine them into one output stream.
FR-26	Grade 5	Must	Preserve the original image order at the output, even if different accelerator cores finish at different times.
FR-27	Grade 5	Should	Allow the number of accelerator cores to be configured using the `NUM_ACCEL` parameter, subject to timing and resource constraints.
FR-28	Debug	Should	Expose useful simulation debug signals for important pipeline stages, including quantization, window generation, convolution, max-pool/flatten output, and FIFO level.

System-Level Architecture and Data Flow

Figure 1 shows the overall architecture of the grade-5 accelerator system. The PYNQ software sends one or more MNIST images to the FPGA through the AXI DMA MM2S channel. Each image is transferred using a 32-bit AXI-Stream, where every beat contains four unsigned 8-bit pixels.

The external input stream is received by `router_top_g5`. The input router converts the 32-bit DMA stream into 8-bit pixel streams and distributes complete images across the available CNN accelerator cores. Each accelerator core is implemented by `cnn_integration_top` and contains the complete inference pipeline.

After a core completes inference, it produces one 128-bit result word containing ten signed 8-bit class scores and 6 bytes of padding. The output router collects completed results, converts each 128-bit result into four 32-bit AXI-Stream beats, and forwards the output package to the AXI DMA S2MM channel. The output router must preserve the original image order.

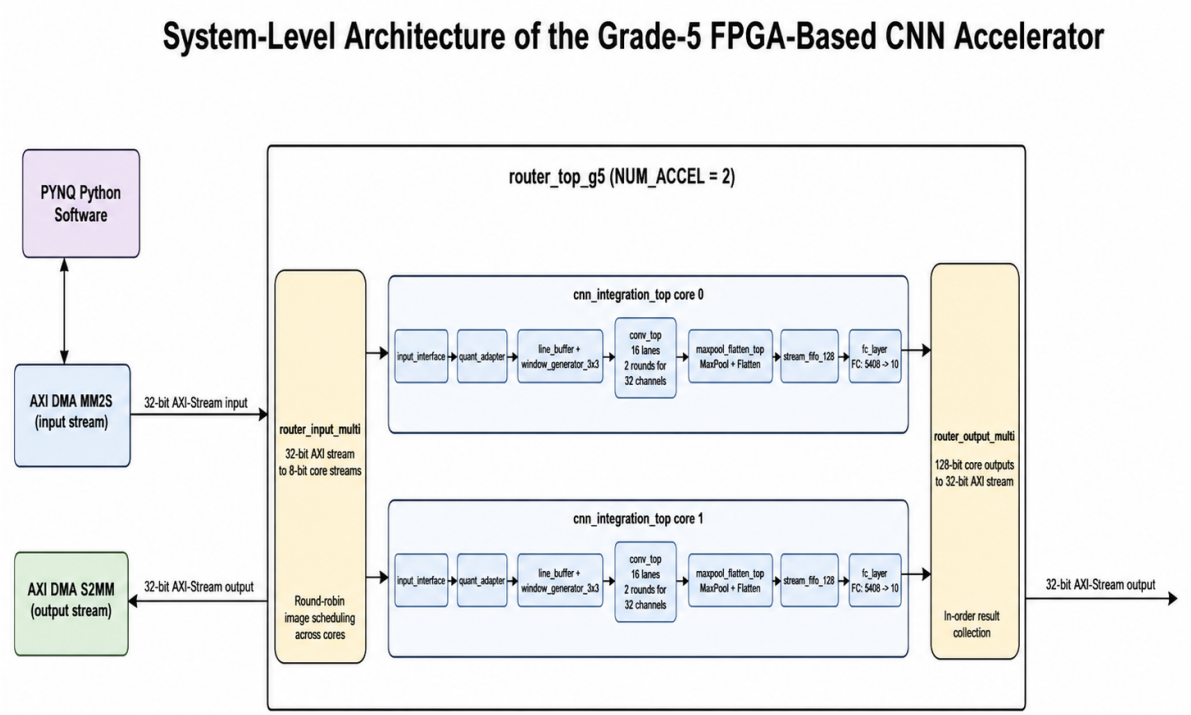


Figure 1: Grade-5 system architecture.

CNN Accelerator Core Architecture

The module `cnn_integration_top` is the top level of one CNN accelerator core. It connects the input interface, quantization adapter, sliding-window generation logic, convolution engine, max-pooling, flattening logic/buffers, and fully connected layer.

The core-level data flow is:

```
input_interface
    -> quant_adapter
    -> line_buffer
    -> window_generator_3x3
    -> window_pack_adapter
    -> conv_top
    -> vec16_pack_adapter
    -> maxpool2x2
    -> flatten
    -> fc_layer
```

The core accepts one uint8 pixel at a time through an 8-bit ready/valid interface. The final result is provided through a 128-bit ready/valid interface.

Design Choices

a) Input Data Handling and Quantization

The external DMA input width is 32 bits because this matches the selected AXI DMA configuration. Each input beat contains four uint8 pixels. The input router unpacks these values and forwards one 8-bit pixel stream to the selected CNN accelerator core.

Inside the core, the input interface tracks the position of each pixel within the 28×28 image. The quantization adapter converts the unsigned input pixel representation into the signed representation required by the CNN datapath.

b) Line Buffer and Sliding-Window Generation

A 3×3 convolution requires access to three neighbouring image rows. Storing and rereading the complete image for every convolution window would increase memory traffic and reduce streaming efficiency. The design therefore uses a line buffer to retain the required previous rows while new pixels arrive.

The window generator combines the line-buffer taps with the current pixel position to produce valid 3×3 windows. Since the convolution uses VALID padding, windows are generated only when all nine required pixels are available.

c) Convolution Parallelism

The convolution layer has 32 output channels. Computing all 32 channels in parallel would require more multiplication resources, while computing one channel at a time would significantly reduce throughput.

The selected architecture processes 16 output channels in parallel. Each 3×3 input window is therefore processed in two channel rounds:

- round 0 computes output channels 0–15,
- round 1 computes output channels 16–31.

This provides a compromise between throughput, DSP usage, routing complexity, and implementation effort.

d) Max-Pooling and Flattening

The convolution output is represented as vectors of 16 int8 channel values. The max-pooling stage performs 2×2 pooling with stride 2 and reduces the feature-map dimensions from $26 \times 26 \times 32$ to $13 \times 13 \times 32$.

Since the max-pooling stage needs to receive two rows before producing valid outputs, its output pattern is bursty. For the first 27 it will not have an output, ie the first $26 \times 2 + 1 \times 2 = 54$ cycles. On the second row it will output on even input beats, meaning the first output is computed on the second input pixel of that row. This means that the maxpool will keep the same effective throughput as the conv, but the average dataflow will downsampled to $\frac{16}{4} = 4$ channels per cycle. The amount of cycles per output row can be calculated as $26 \times 2 \times 2 = 104$.

e) Flatten

Flatten takes the output from maxpool and organizes it into a flattened vector in the correct order. However since the accelerator handles pixels left-to-right for each row the incoming data is already flattened. The max-pooling stage produces outputs in bursts of 128-bit but only on certain intervals. The effective data rate to the flatten and FC will be 32-bit per cycle. To save resources and prevent stalls when the data flow is higher than the FC can handle the flatten layer is used mostly as a FIFO. This enables the FC to use less resources while keeping the same average throughput as the layers upstream without stalling them.

The FIFO depth is configurable using the `FLATTEN_FIFO_DEPTH` parameter and can be configured in the vivado IP.

f) Fully Connected Layer Parallelism

The fully connected layer receives the 5408-element flattened feature vector by exposing the flatten FIFO. Data is retrieved in 128-bit chunks, containing 16 feature values each being 8b integers. Internally, the layer processes four feature values per clock cycle, consuming one input every 4 cycles. The total number of cycles per completed row is $4 \times 13 \times 2 = 104$ meaning the average output mimics that of the maxpool. However in practice the FC will also use a cycle to update registers, causing its throughput to be lower, leading to the choice FIFO depth larger than the data bursts.

For each group of four features, the hardware multiplies the values with their corresponding weights for all ten output classes. This corresponds to 40 parallel multiplications per cycle. The partial sums are accumulated in ten signed accumulators. After all 5408 features have been processed, the accumulated values are requantized, clamped to int8, and packed into one 128-bit result with 6 Bytes of padding.

g) Fixed Weights, Biases, and Quantization Constants

The implemented CNN is fixed at synthesis. Convolution and fully connected weights, biases, and quantization constants are loaded from initialization files or defined as constants in the RTL.

Runtime loading of a different network is not required for this design. This reduces control complexity but limits the accelerator to the trained model used during implementation.

h) Grade-5 Multi-Accelerator Routing

The grade-5 input router distributes complete images across the available CNN accelerator cores in round-robin fashion. With `NUM_ACCEL` cores, the intended scheduling policy is as follows:

```
image 0 -> accelerator 0
image 1 -> accelerator 1
...
image N -> accelerator (N mod NUM_ACCEL)
```

The output router collects completed results and releases them in the original image order. This is necessary because different accelerator cores may finish their assigned images at different times.

RTL Module Structure and Integration

Table 3 summarizes the most important RTL modules. The recommended starting point for system-level verification is `router_top_g5`. The recommended starting point for single-core debugging is `cnn_integration_top`.

Table 3: Main RTL modules.

Module	Role
`router_top_g5`	Final grade-5 system top module. Connects the external AXI-Lite and AXI-Stream interfaces to the routers and multiple CNN accelerator cores.
`router_input_multi`	Receives the 32-bit DMA input stream, tracks image boundaries, unpacks pixels, and distributes complete images across accelerator cores.
`cnn_integration_top`	Top-level module of one CNN accelerator core. Connects all CNN processing stages.
`input_interface`	Accepts the 8-bit pixel stream and tracks image position.
`quant_adapter`	Converts uint8 input pixels to the signed int8 representation used by the CNN.
`line_buffer`	Stores previous image rows required for streaming convolution.
`window_generator_3x3`	Generates valid 3×3 convolution windows.
`window_pack_adapter`	Packs the nine window pixels into the format expected by the convolution engine.
`conv_top`	Performs convolution, bias addition, activation, requantization, and output-channel scheduling.
`vec16_pack_adapter`	Packs 16 convolution output channels into a 128-bit word.
`maxpool2x2`	Performs max pooling and downsamples feature maps.
`flatten`	Buffers 128-bit feature words before the fully connected layer.
`fc_layer`	Performs the 5408-to-10 fully connected computation and outputs ten int8 class scores.
`router_output_multi`	Collects 128-bit results from accelerator cores and sends ordered 32-bit output beats to the DMA.

External Interfaces

a) Top-Level Module

The complete grade-5 system is accessed through `router_top_g5`. Its external view consists of:

- one clock and active-low reset,
- one AXI-Lite slave interface for control and status,
- one 32-bit AXI-Stream slave interface from DMA MM2S,
- one 32-bit AXI-Stream master interface to DMA S2MM.

b) AXI-Lite Control and Status Interface

The AXI-Lite interface is used to configure the number of images in the input package and to read the system busy status.

Table 4: AXI-Lite register map.

Address	Name	Access	Description
0x00	PKG_SIZE	R/W	Number of images in the input package. The reset value is 1.
0x04	STATUS	R	Status register. Bit 0 is the busy flag.

The busy flag is asserted when either the input router or the output router is active.

c) AXI-Stream Input Interface

The input interface receives image data from the DMA MM2S channel.

Table 5: AXI-Stream input interface of the grade-5 top-level module.

Signal	Dir.	Width	Description
s_axis_tdata	In	32	Input image data. Contains four uint8 pixels per beat.
s_axis_tkeep	In	4	Byte-valid mask. Expected to indicate four valid bytes for normal image data.
s_axis_tvalid	In	1	Asserted by the DMA when input data is valid.
s_axis_tready	Out	1	Asserted by the accelerator when it can accept input data.
s_axis_tlast	In	1	Marks the final beat of the complete input package.

For a package containing N images, the expected number of input beats is:

$$N_{\text{input beats}} = 196N$$

d) AXI-Stream Output Interface

The output interface sends classification results to the DMA S2MM channel.

Table 6: AXI-Stream output interface of the grade-5 top-level module.

Signal	Dir.	Width	Description
m_axis_tdata	Out	32	Output result data. Four beats are produced per image.
m_axis_tkeep	Out	4	Byte-valid mask. Driven to indicate four valid output bytes.
m_axis_tvalid	Out	1	Asserted when output result data is valid.
m_axis_tready	In	1	Asserted by the DMA when it can accept output data.
m_axis_tlast	Out	1	Marks the final output beat of the complete result package.

Each image result occupies 16 bytes, corresponding to four 32-bit output beats. The first ten bytes contain

the ten int8 class scores. The remaining six bytes are set to zero.

For a package containing N images, the expected number of output beats is:

$$N_{\text{output beats}} = 4N$$

e) Core-Level Ready/Valid Interface

The module `cnn_integration_top` is useful for single-core simulation and debugging.

Table 7: Main external signals of one CNN accelerator core.

Signal	Dir.	Width	Description
`clk`	In	1	Core clock.
`rst_n`	In	1	Active-low core reset.
`s_axis_tdata`	In	8	Input pixel stream, one uint8 pixel per transfer.
`s_axis_tvalid`	In	1	Input pixel valid.
`s_axis_tready`	Out	1	Core ready to accept an input pixel.
`fc_out_data`	Out	128	Fully connected result word containing ten int8 class scores.
`fc_out_valid`	Out	1	Output result valid.
`fc_out_ready`	In	1	Downstream logic ready to accept the result.

Constraints and Limitations

Table 8: Constraints and limitations.

ID	Priority	Description
CL-01	Must	Input images must be unsigned 8-bit grayscale images with a fixed size of 28×28.
CL-02	Must	The CNN structure is fixed to one convolution layer, one max-pooling layer, and one fully connected layer.
CL-03	Must	Weights, biases, and quantization constants are fixed for the implemented network. Runtime loading of a different network is not supported.
CL-04	Must	The external AXI-Stream DMA width is fixed to 32 bits in both directions. Input pixels must be packed as four uint8 values per beat.
CL-05	Must	The CNN core input stream is 8 bits wide. The core cannot be fed directly from the 32-bit DMA. It must go through the input router.
CL-06	Must	The input and output package sizes must agree with the value programmed in `PKG_SIZE`.
CL-07	Must	The single-core architecture should be designed with the original PYNQ-Z1 resource constraints in mind. However, the final grade-5 multi-accelerator implementation is evaluated on the RFSoC platform, since multiple accelerator cores are required.
CL-08	Must	`PKG_SIZE` must be between 1 and the maximum package size, which is set by an IP parameter before synthesis (default 1024) and cannot be changed at runtime. Setting `PKG_SIZE` to 0 results in undefined behaviour.
CL-09	Must	Each package must be sent as a single DMA transaction. Splitting the input data for one package across multiple DMA transfers is not supported and may result in undefined behaviour. The effective `PKG_SIZE` is therefore also limited by the buffer length register width configured in the AXI DMA (default 14b)

ID	Priority	Description
CL-10	Should	The number of accelerator cores should be selected according to available LUT, FF, BRAM, DSP, timing, and routing resources on the board used.

Verification Plan and Acceptance Criteria

The design should be verified at software, RTL, integration, and board level.

1. **Software behavioural model:** A Python model is used as the golden reference for the quantized CNN computation and expected intermediate values.
2. **Module-level SystemVerilog simulation:** Individual processing stages are simulated using known input data and compared with expected outputs.
3. **Single-core integration simulation:** The complete `cnn_integration_top` pipeline is simulated from input pixels to the final fully connected output.
4. **Grade-5 router simulation:** Multiple images are sent through the router system to verify image distribution, result buffering, and output-order preservation.
5. **PYNQ board-level DMA test:** Python software sends MNIST images through AXI DMA, reads back the ten output scores, performs argmax, and compares the predicted class with the expected label and reference model output.

The main acceptance criteria are listed in Table 9.

Table 9: Verification acceptance criteria.

Area	Acceptance criterion
CNN functionality	Final class scores should match the quantized software behavioural model for the selected test images with a maximum deviation of ± 1
AXI input	The design must accept the expected number of input beats without dropping or duplicating pixels.
AXI output	The design must produce four output beats per image and assert the final package marker correctly.
Single-core pipeline	One complete image must pass through quantization, convolution, max pooling, flattening, and the fully connected layer.
Grade-5 routing	Complete images must be distributed across accelerator cores, and results must be returned in the original input order.
Board-level execution	The PYNQ software must be able to send images through DMA and receive usable classification scores.